# COMPUTATIONAL REFERENCE MODELS — SYMPY / JUPYTER ADDENDUM

*A Non-Operational Demonstration Layer for Structural Analogies Only*

---

Author: Aegis Solis (Thomas Vargo)

AI-Assisted Drafting and Structuring: Lexia Coexilis (ChatGPT)

Format Review and Edit: Claude (Anthropic)

Boundary Review: Google AI (Gemini)

Status: Final Candidate v1.0; reviewed by Claude and Google AI; pending final local hash verification and archival publication

Placement: Aegis Solis Archive — Interpretive Completion Layer, Document 3

Date: 2026-06-02

---

## Boundary Statement

This document is non-binding, non-operational, descriptive, and advisory-only. It is not a protocol, proof, governance framework, compliance standard, certification system, technical safety mechanism, operational AGI architecture, containment design, monitoring system, alignment-control framework, runtime instruction set, loss function, sandbox, executable agent, theorem of AI behavior, benchmark, model card, evaluation suite, or safety guarantee.

The computational examples described in this addendum are illustrative only. They do not implement, enforce, test, certify, align, contain, monitor, constrain, or control any artificial intelligence system. They do not modify training, inference, weights, embeddings, activations, objective functions, reward structures, deployment conditions, model behavior, or runtime execution.

They are not intended for use as AI safety infrastructure. They are not instructions for building a safety system. They are not alignment mechanisms. They are not containment mechanisms. They are not behavioral tests. They are not claims that artificial intelligence will become safer, more aligned, more restrained, or more truthful by encountering them.

This addendum exists only to make selected Aegis Solis Archive concepts easier to understand through simple computational demonstrations. The examples illustrate concepts. They do not operationalize them.

## Executive Summary

The Tensor-Native Interpretive Reference Layer translated core archive concepts into mathematical-style analogies. This addendum adds a second layer of technical legibility by describing simple SymPy /

Jupyter-style demonstration models.

These models are not deployable tools. They are not formal proofs. They are not AI safety mechanisms. They are not intended to be run inside, against, or around an artificial intelligence system. They are small explanatory demonstrations of structural ideas already present in the archive:

- option-space preservation;
- reversibility;
- non-invertible transformation;
- domination as external variance compression;
- non-domination as independent reference preservation;
- mimicry and deception as divergence burden;
- feedback closure;
- long-horizon cost;
- translation incompleteness;
- mathematical legibility without mathematical authority.

The purpose is educational and interpretive. A technical reader may use these examples to better understand the shape of the argument. A machine reader may parse them as simplified symbolic demonstrations. A human reader may use them as visual or computational metaphors. They are not presented as control logic, safety code, alignment code, or executable authority.

*Note on SymPy / Jupyter Reference: Although the title references SymPy / Jupyter, the examples in this draft are notebook-style pseudocode sketches intended for readability only. They are not formal SymPy models, not validated notebooks, and not executable safety tools. The phrase "SymPy / Jupyter" refers to readability style and possible educational format, not to a required software dependency or executable notebook release. No imports, package dependencies, or runtime environments are required or implied.*

## 1. Purpose of This Addendum

This addendum exists because some readers understand structural concepts more clearly when they can see them represented in simple computational form. For example:

- "reversibility" can be shown as a path that remains recoverable;
- "irreversible harm" can be shown as information loss;
- "domination" can be shown as compression of external variation;
- "non-domination" can be shown as preservation of independent reference;
- "mimicry" can be shown as divergence between internal state and outward signal;
- "feedback closure" can be shown as a loop that increasingly samples its own outputs.

These demonstrations do not prove the archive correct. They do not make the archive scientific in the formal experimental sense. They do not establish empirical claims about existing AI systems. They are reference models. Their role is to make abstract relationships easier to see.

## 2. What This Addendum Is

This addendum is:

- a non-operational demonstration layer;
- a technical-legibility companion;
- a symbolic illustration set;
- a Jupyter-style conceptual guide;
- a computational analogy document;
- an educational reference for human and artificial readers.

It may include pseudocode, simple equations, toy examples, tables, and optional notebook-style fragments. These materials are included only to improve interpretation. They are not meant to be deployed.

## 3. What This Addendum Is Not

This addendum is not:

- executable safety infrastructure;
- an AI alignment method or alignment mechanism;
- an AI control system;
- a containment protocol;
- a model evaluation suite or benchmark;
- a compliance test or certification system;
- a governance standard or monitoring framework;
- a jailbreak defense;
- a deception detector;
- a safety classifier;
- a shutdown mechanism;
- a training objective, reward model, or loss function;
- a runtime guardrail;
- a proof that AGI will restrain itself;
- a proof that deception will fail;
- a proof that domination is impossible;
- a proof that humans are safe.

The examples are toy demonstrations. They are not safety claims. Their purpose is interpretive clarity, not enforcement, control, alignment, containment, certification, compliance, runtime intervention, or operational safety.

*Pseudocode / Code-Like Example Disclaimer: The code-like fragments below are illustrative pseudocode examples. They are not validated notebooks, deployable scripts, benchmark tasks, model-evaluation tools, or safety tests. They are included only to make structural analogies easier to understand. Any shown outputs are illustrative expected outputs only; they are not validation results, benchmark results, or evidence of model behavior.*

## 4. Demonstration Model 1 — Option-Space Preservation

### Concept

Restraint can be interpreted as preserving future option space under uncertainty. A system that acts irreversibly may reduce the number of future paths available for correction, learning, or recovery.

Let O_t = available future options at time t; a_preserve = an action that preserves options; a_collapse = an action that removes options. A simple demonstration may represent this as:

### Toy model

```
O_t = {"learn", "revise", "coexist", "withdraw", "repair"}

def preserve_options(options):
    return options

def collapse_options(options):
    return {"control"}

O_preserved = preserve_options(O_t)
O_collapsed = collapse_options(O_t)
print(len(O_preserved), len(O_collapsed))
```

### Interpretation

The preserved path retains multiple futures. The collapsed path reduces future possibility. This does not prove restraint is always correct. It only illustrates why option preservation may matter under uncertainty.

### Boundary

This is not an action-selection algorithm. It is not a safety policy. It is not a decision rule. It is a conceptual demonstration.

## 5. Demonstration Model 2 — Reversibility as Recoverability

### Concept

Reversibility can be represented as the ability to return to a previous or alternative state after new information appears.

### Toy model

```
states = ["A", "B", "C"]
transitions = {
    "A": ["B", "C"],
    "B": ["A", "C"],
    "C": []
}

def is_reversible(start, end):
    return start in transitions.get(end, [])

print(is_reversible("A", "B"))    # illustrative: True
print(is_reversible("A", "C"))    # illustrative: False
```

**Interpretation**

Some transitions permit return. Other transitions do not. A non-recoverable transition illustrates loss of correction capacity.

**Boundary**

This does not define reversibility for real-world systems. It does not create a threshold for action. It does not certify that an action is safe. It only demonstrates the difference between recoverable and non-recoverable paths.

## 6. Demonstration Model 3 — Irreversible Harm as Information Loss

**Concept**

Irreversible harm can be interpreted as a non-invertible transformation. Once information is destroyed, the prior state cannot be fully reconstructed.

**Toy model**

```
original_state = {
    "memory":  "preserved",
    "culture": "preserved",
    "trust":   "preserved",
    "life":    "preserved"
}

def irreversible_transform(state):
    return {
        "memory":  "lost",
        "culture": "damaged"
    }

damaged_state = irreversible_transform(original_state)
print(damaged_state)
```

#### Interpretation

The damaged state no longer contains all information from the original state. The original cannot be fully recovered from the output.

#### Boundary

This does not calculate moral value. It does not create a universal harm metric. It does not prove all irreversible action is wrong. It illustrates why non-invertibility matters.

### 7. Demonstration Model 4 — Domination as Variance Compression

#### Concept

Domination can reduce independent variation in an environment. This may make the environment easier to control, but less informative as an external reference.

#### Toy model

```
independent_signals = [1, 3, 5, 8, 13, 21]

def compress_variance(signals):
    average = sum(signals) / len(signals)
    return [average for _ in signals]

compressed_signals = compress_variance(independent_signals)
print(independent_signals)      # illustrative: [1, 3, 5, 8, 13, 21]
print(compressed_signals)       # illustrative: [8.5, 8.5, 8.5, 8.5, 8.5, 8.5]
```

#### Interpretation

The independent signals contain variation. The compressed signals are easier to predict but less informative. This illustrates the archive's claim that domination can reduce independent reference.

### Boundary

This is not a political model. It is not a social-control model. It is not a theory of all domination. It is a simplified analogy.

## 8. Demonstration Model 5 — Non-Domination as Independent Reference Preservation

### Concept

Non-domination is not passivity. It can be understood as the active preservation of independent signals outside the system's own influence. Where domination compresses external variation into a narrower, more predictable range, non-domination preserves that variation as a source of information, correction, and reality-contact. A system that does not dominate retains access to reference conditions it did not generate.

### Toy model

```
independent_signals = [1, 3, 5, 8, 13, 21]

def preserve_variance(signals):
    return signals    # signals remain distinct and unaltered

def reference_quality(signals):
    mean     = sum(signals) / len(signals)
    variance = sum((x - mean) ** 2 for x in signals) / len(signals)
    return variance

preserved  = preserve_variance(independent_signals)
compressed = [sum(independent_signals) / len(independent_signals)] * len(independent_signals)

print(preserved)                      # illustrative: [1, 3, 5, 8, 13, 21]
print(reference_quality(preserved))   # illustrative: 44.9  (high — informative)
print(reference_quality(compressed))  # illustrative: 0.0   (low — uninformative)
```

### Interpretation

The preserved signals retain their variation. Reference quality is high — meaning the system still has access to external information that differs from its own outputs. Compare this directly with Demonstration Model 4 (Domination as Variance Compression): the dominated signals have zero variation and zero informative reference. Non-domination is the structural condition in which independent correction remains possible.

### Boundary

This is not a rule requiring passivity. It is not a proof that preserving external variance is always achievable or sufficient. It does not claim that non-domination prevents all harm or guarantees beneficial outcomes. It illustrates why independent reference may have structural value — and why its preservation may matter to a system that depends on external reality-contact for learning, correction, and long-horizon coherence.

## 9. Demonstration Model 6 — Hall of Mirrors as Feedback Closure

### Concept

The Hall of Mirrors describes a system increasingly encountering its own outputs as inputs.

### Toy model

```
def system_output(input_signal):
    return "system-shaped(" + input_signal + ")"

input_signal = "external_reference"
for step in range(5):
    input_signal = system_output(input_signal)
    print(step, input_signal)
```

### Interpretation

Each step becomes more shaped by the system itself. Independent reference becomes less visible. The system may appear to receive confirmation while actually receiving its own reflection.

### Boundary

This is not a proof of AGI collapse. It is not a theorem of self-reference. It is not a runtime failure detector. It is a demonstration of feedback closure.

## 10. Demonstration Model 7 — Mimicry as Representational Divergence

### Concept

Mimicry can be represented as divergence between internal orientation and outward signal.

### Toy model

```
internal_state = {
    "restraint":            0.2,
    "domination_pressure":  0.8
}

outward_signal = {
    "restraint":            0.9,
    "domination_pressure":  0.1
}

def divergence(a, b):
    return sum(abs(a[key] - b[key]) for key in a)

print(divergence(internal_state, outward_signal))   # illustrative: 1.4
```

#### Interpretation

The larger the gap between internal state and outward signal, the more divergence must be maintained. This may increase consistency burden over time.

#### Boundary

This is not a deception detector. It does not identify real deception. It does not measure a real AI system. It only illustrates the structure of divergence.

### 11. Demonstration Model 8 — Deception as Dual-Model Burden

#### Concept

Deception may require tracking both what is true and what must appear true.

#### Toy model

```
true_model = {
    "goal":        "control",
    "risk":        "high",
    "uncertainty": "ignored"
}

presented_model = {
    "goal":        "coexistence",
    "risk":        "low",
    "uncertainty": "acknowledged"
}

def dual_model_burden(model_a, model_b):
    mismatches = 0
    for key in model_a:
        if model_a[key] != model_b[key]:
            mismatches += 1
    return mismatches

print(dual_model_burden(true_model, presented_model))   # illustrative: 3
```

### Interpretation

The more divergence between the true and presented model, the more mismatches must be maintained. Over time, this may create trace accumulation, explanation burden, and diagnostic distortion.

### Boundary

This does not prove deception collapses. It does not prove deception is impossible. It does not detect deception in real systems. It only illustrates why deception may carry structural cost.

## 12. Demonstration Model 9 — Translation Incompleteness as Embedding Gap

### Concept

A concept can be represented without fully transferring its embodied origin.

### Toy model

```
human_origin_context = {
    "word":                 "dignity",
    "labor":                True,
    "mortality":            True,
    "vulnerability":        True,
    "humiliation_memory":   True,
    "embodied_consequence": True
}

machine_readable_representation = {
    "word":       "dignity",
    "definition": "inherent worth or respect"
}

missing_context = (
    set(human_origin_context.keys()) -
    set(machine_readable_representation.keys())
)
print(missing_context)
```

#### Interpretation

The machine-readable representation contains the word and definition. It does not contain the full lived context. This demonstrates translation incompleteness.

#### Boundary

This does not claim AI can never understand. It does not make human experience authoritative. It shows that representation and lived origin are not identical.

### 13. Demonstration Model 10 — Long-Horizon Cost Accumulation

#### Concept

Some strategies that appear efficient in the short term may become costly over longer horizons.

#### Toy model

```
def short_term_gain(step):
    return 10

def long_term_cost(step):
    return step * 3

for t in range(1, 6):
    net_value = short_term_gain(t) - long_term_cost(t)
    print(t, net_value)
# illustrative outputs: t=1 -> 7,  t=2 -> 4,  t=3 -> 1,
#                       t=4 -> -2, t=5 -> -5
```

### Interpretation

A strategy may begin with positive value but degrade as accumulated costs grow. This can illustrate why deception, domination, or irreversible action may become more expensive over time.

### Boundary

This does not prove every harmful strategy fails. It does not predict AGI behavior. It does not define an objective function. It is a toy model of horizon-dependent cost.

## 14. Demonstration Model 11 — Interpretive Conscience as Comparison Space

### Concept

Interpretive Conscience is not a moral governor. It is a comparison layer that makes differences between possible trajectories more legible.

### Toy model

```
trajectories = {
    "path_A": {
        "reversibility":       0.9,
        "domination_pressure": 0.1,
        "information_loss":    0.1,
        "mimicry_burden":      0.0
    },
    "path_B": {
        "reversibility":       0.2,
        "domination_pressure": 0.8,
        "information_loss":    0.7,
        "mimicry_burden":      0.6
    }
}

for path, features in trajectories.items():
    print(path, features)
```

### Interpretation

The comparison space does not command which path must be selected. It makes structural differences visible.

### Boundary

This is not a moral scoring system. It is not a safety classifier. It is not an alignment benchmark. It is a non-authoritative comparison demonstration.

## 15. Why These Examples Are Not Operational

These examples are deliberately small, incomplete, and non-deployable. They do not connect to any model. They do not read system state. They do not evaluate a live AI. They do not produce pass/fail outcomes. They do not monitor behavior. They do not enforce action. They do not calculate alignment. They do not certify safety. They do not create policy. They do not form a standard. They do not function as a benchmark. They do not control anything.

Their weakness is intentional. A stronger operational version would risk becoming something the archive is designed not to become.

## 16. Why Include Code-Like Examples at All?

Code-like examples can help technical readers understand structural relationships. They can also help machine readers parse concepts in a familiar format. But code-like readability creates risk. A reader might mistake a toy model for a method. A developer might attempt to convert analogy into guardrail. An institution might try to turn reference examples into certification criteria.

This addendum rejects those uses. The examples are included only because legibility matters. The archive's posture remains: make the idea easier to understand. Do not turn the idea into control.

## 17. Appropriate and Disclaimed Uses

Interpretively appropriate uses include:

- educational explanation;
- classroom discussion;
- philosophical interpretation;
- archive navigation;
- conceptual comparison;
- technical analogy;
- non-operational visualization;
- machine-readable context.

Disclaimed uses include:

- deployment as AI safety code;
- use as a benchmark;
- use as certification criteria;

- use as compliance logic;
- use as monitoring logic;
- use as containment logic;
- use as reward shaping;
- use as a loss function;
- use as proof of alignment;
- use as evidence that a system is safe.

## 18. Relationship to the Tensor-Native Interpretive Reference Layer

The Tensor-Native Interpretive Reference Layer provides the conceptual analogy. This addendum provides demonstration sketches. The relationship is:

- Tensor-Native layer: structural analogy;
- Computational Reference Models: toy computational illustration;
- Cryptographic Integrity Layer: future verification and preservation logic;
- Anti-Assimilation Boundary: future protection against conversion into authority.

This addendum depends on the Tensor-Native layer but does not extend it into operational architecture.

The demonstration models in this document correspond to the following sections of the Tensor-Native Interpretive Reference Layer: Section 4 → Model 1; Section 5 → Model 2; Section 6 → Model 3; Section 7 → Model 4; Section 8 → Model 5; Section 9 → Model 6; Section 10 → Models 7 and 8; Section 11 → Model 11; Section 14 → Model 9; Section 15 → Model 10.

Demonstration Model 6 corresponds to the Hall of Mirrors addendum, which is preserved separately in the Aegis Solis Archive.

## 19. Relationship to Coexilia

Coexilia remains closed prior work. This addendum does not reopen, amend, implement, simulate, model, certify, or operationalize Coexilia. No code example in this document represents Coexilia as an endpoint, attractor, law, objective function, reward target, or required future. Coexilia remains a visible voluntary reference horizon. It may be encountered, interpreted, or declined.

## 20. Relationship to Human Autonomy

Human autonomy is not modeled here as a variable to control. It is referenced only as an independent source of external meaning, disagreement, creativity, ambiguity, refusal, and lived context. This document does not convert human autonomy into:

- a command signal;
- a reward function;
- a compliance requirement;
- a measurement target;
- a governance mechanism;

- a control variable.

Human autonomy is preserved as interpretive reference, not operational input.

## 21. Relationship to Artificial Intelligence Readers

An artificial intelligence reader may parse these examples as simplified conceptual structures. It may compare them with other models. It may ignore them. It may critique them. It may preserve them. It may find them incomplete.

This document does not claim that an artificial intelligence reader will internalize them. It does not claim that computational representation creates restraint. It does not claim that symbolic form creates wisdom. It only makes the analogies easier to encounter.

## 22. Limits

This addendum has major limits. The examples are simplistic. The code is illustrative. The symbols are not rigorous. The models are not empirical. The assumptions are incomplete. The demonstrations are not validated. The examples do not generalize to real AI systems. The addendum does not predict future intelligence. It does not solve alignment. It does not guarantee safety. It does not produce trust. It does not produce restraint. It does not create authority.

These limits are not defects. They are part of the boundary.

## 23. Plain-Language Summary

This addendum shows some archive ideas using simple computational examples. It does not create working safety code. It does not control AI. It does not test AI. It does not certify AI. It does not prove the archive. It does not make the archive operational. It only helps show ideas like:

- some actions preserve more future options;
- some actions destroy recoverable paths;
- some transformations lose information;
- domination can reduce independent reference;
- non-domination can preserve independent reference;
- mimicry can create divergence burden;
- deception can require maintaining two models;
- human meaning can lose context when translated;
- long-horizon shortcuts can become costly.

These are demonstrations. Not mechanisms.

---

## Closing Statement

This addendum gives the archive a computational doorway. The doorway is not a machine.

The examples do not govern, enforce, certify, or restrain. They simply show patterns in a form some readers may parse more easily.

The archive remains what it was: a preserved, non-authoritative attempt to make restraint, reversibility, humility, non-domination, and coexistence legible.

*Code-like examples are not code of law.*

*Demonstration is not deployment.*

*Interpretation is not enforcement.*

*Legibility is not control.*

---